Claims Integrity and Risk Analysis

Prepared for: Health Insurance Claims Department and Analytics Teams

---

Technical Report

Deliverables:

- Three key insights about claims patterns, costs, or risk
- Fraud detection logic
- One actionable recommendation for reducing overspending or improving claims processing
- SQL script with detailed comments explaining validation and detection logic

Insight 1: Identify High-Cost Areas

To help us understand financial risks linked to different treatment types, it is important to identify high-cost areas. The result showed **Chemotherapy** incurring the most cost, from 49 appointments, across five different treatment types, whereas **MRI** incurred the most cost on average from a total of 36 appointments.

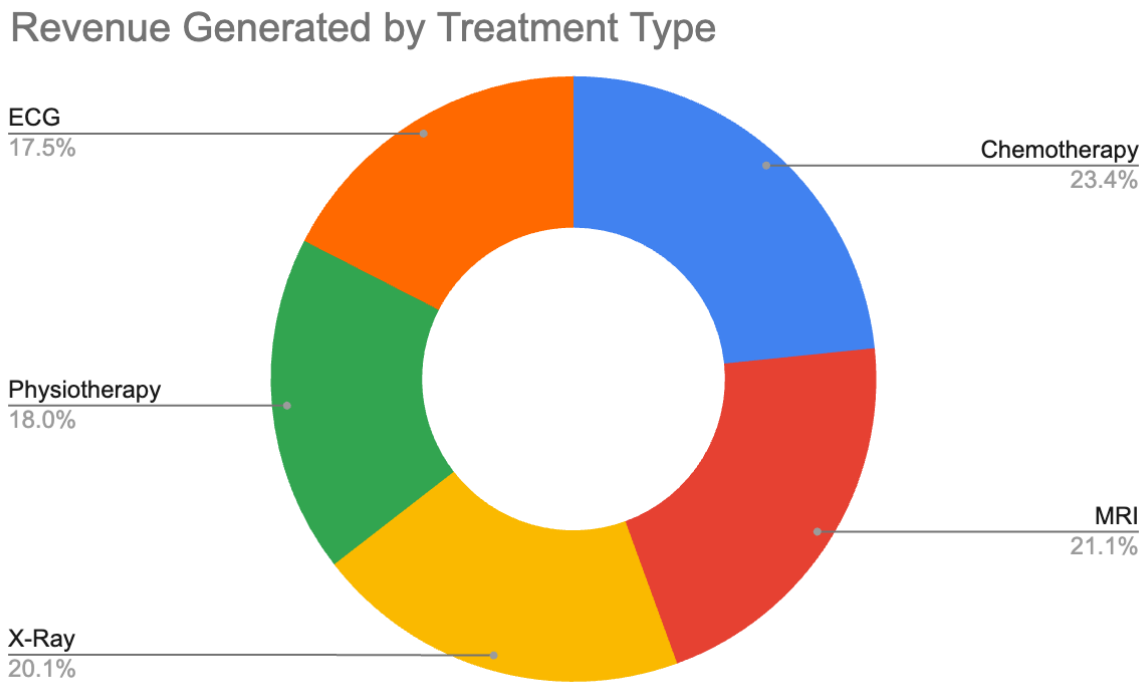

Figure 1

The above chart helps us break down each treatment type's coverage of total revenue generated. This helps us in further analysing treatment types to see whether the revenues generated by them are legitimate.

We checked how much revenue was lost due to payments that have **Failed** status. We can see from the figure below, **Physiotherapy** and **Chemotherapy** are the two treatment types that have incurred the most loss in revenue.

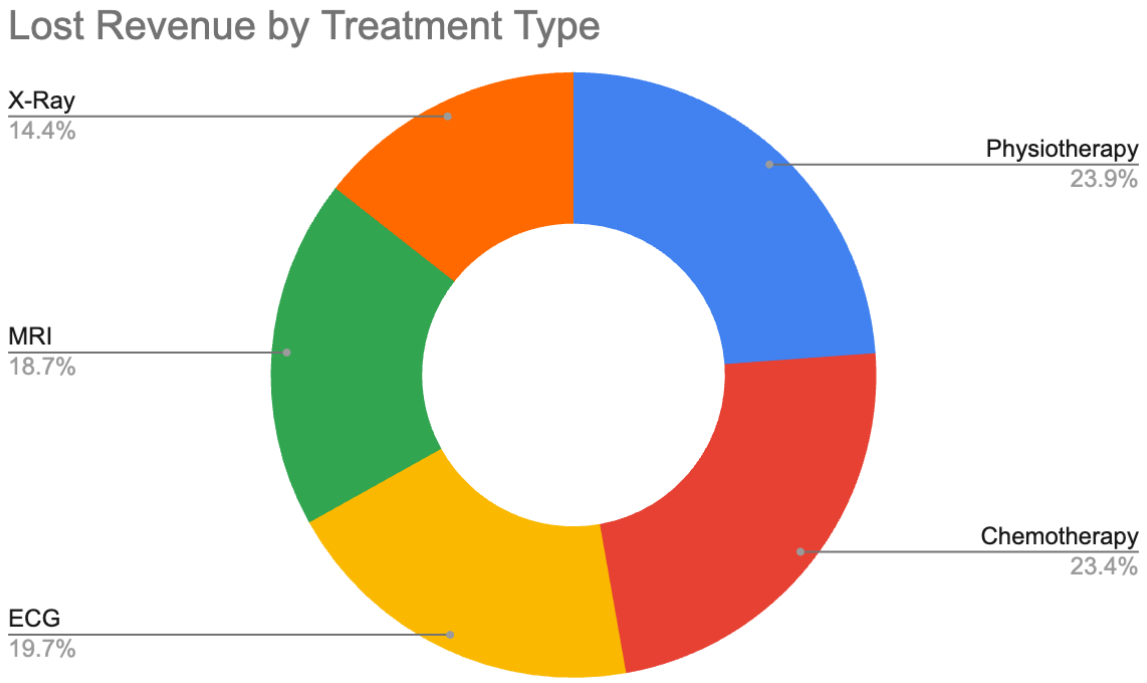

Figure 2

Overall, we learned that the total revenue lost was 35.05% of the total revenue generated.

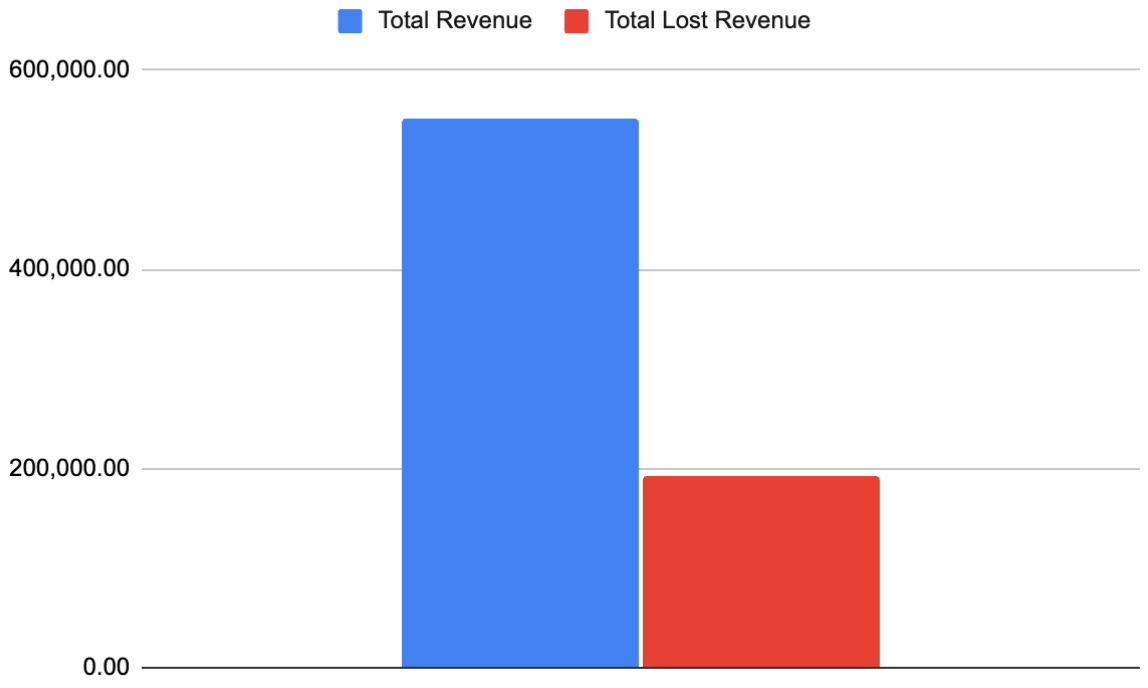

Figure 3

This is a significant portion of revenue lost due to failed payments. The claims department needs to investigate why and how this has come about. At the same time they need to come up with other methods to recover this.

Insight 2: Claims Patterns

We need to verify claims accuracy, which is vital to enhancing trust between patients and healthcare providers. Verifying claims accuracy also helps us in reducing risks of audits and penalties by complying with legal medical regulators.

Upon analysing the appointments table, it was noticed that out of 200 appointments 103 were either cancelled or were no-shows. That is more than 50% of appointments that were not fulfilled. The figure below illustrates that.

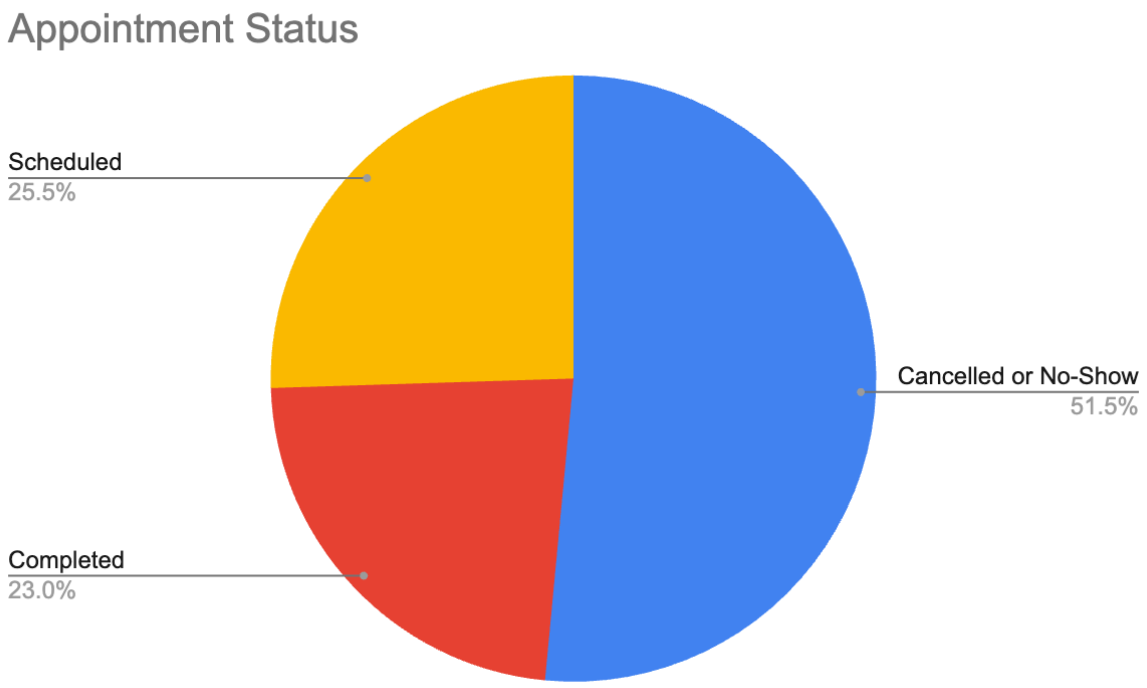

Figure 4

Upon further investigation, it was discovered that 81 out of 103 appointments' payment status had been set to paid. This shows the billing department had approved payments for appointments that had not been completed.

Row	appointment_id	status	payment_status
1	A002	No-show	Paid
2	A003	Cancelled	Paid
3	A008	Cancelled	Paid
4	A014	Cancelled	Paid
5	A015	No-show	Paid
6	A019	Cancelled	Paid
7	A021	No-show	Paid
8	A022	No-show	Paid

Results per page: 50 1 – 50 of 81

Figure 5

Insight 3: Patient Risk Analysis

It is important to look into risks brought on by patients as any unnoticed data might end up costing organisations time and money.

We looked into patient distribution according to gender. From our dataset of 50 patients, 38% were female whereas 62% were male.

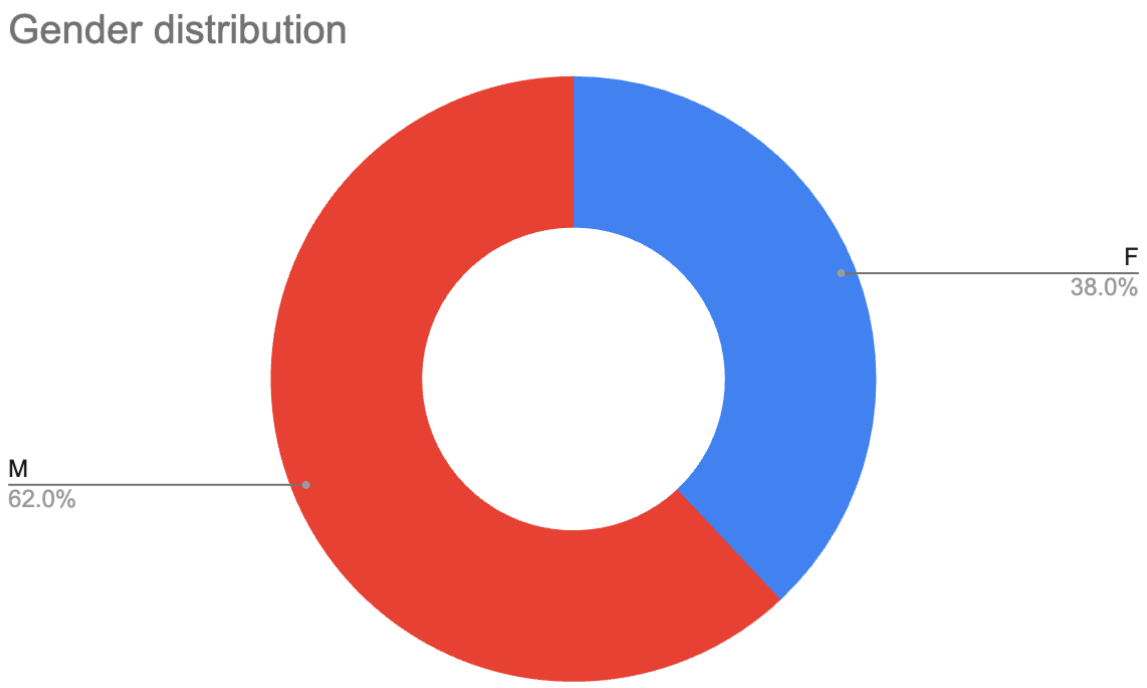

Figure 6

We then grouped patients by age into bands (18 - 24, 25 - 34, 35 - 44, 45 - 54, 55 - 64, 65+) assuming all patients are above 18 years of age. We noticed the 25 - 34 age band made up the most number of patients, whereas the 55 - 64 age band made the least.

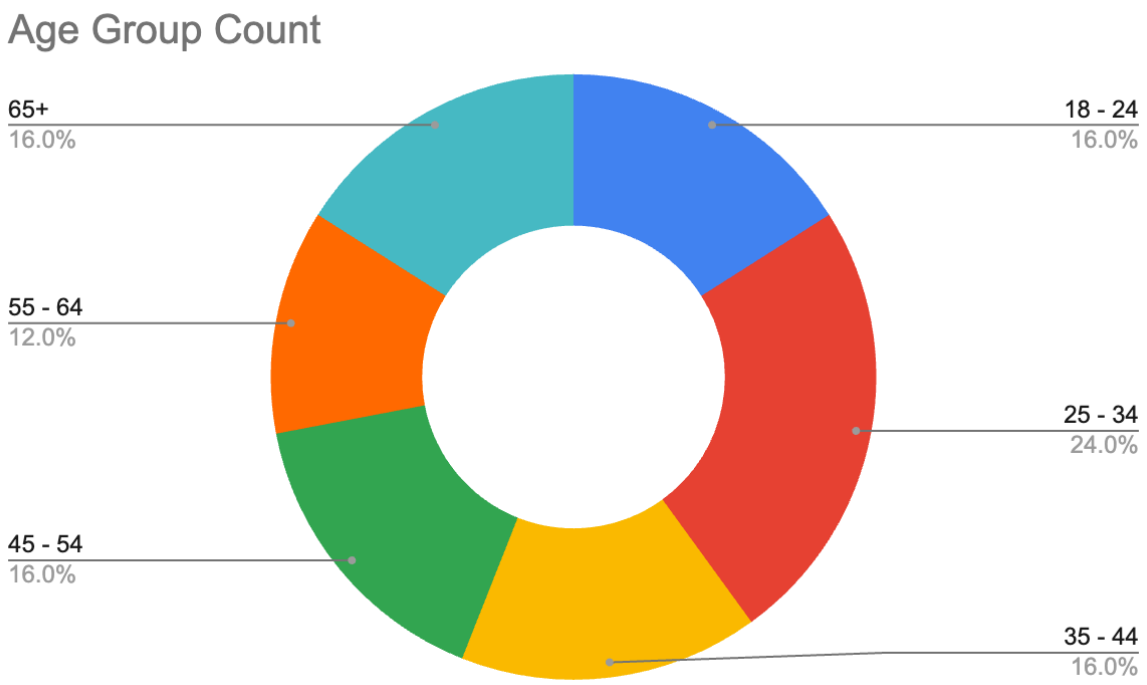

Figure 7

When we looked further into how each patient's appointment records was, we found some very interesting figures. Patient **P012** in particular had the strangest case where they had ten appointments in total, but none of those appointments were completed. Six of them were cancelled and four of them were no-whos. This gives us a picture of a patient who has cost the organisation time and money.

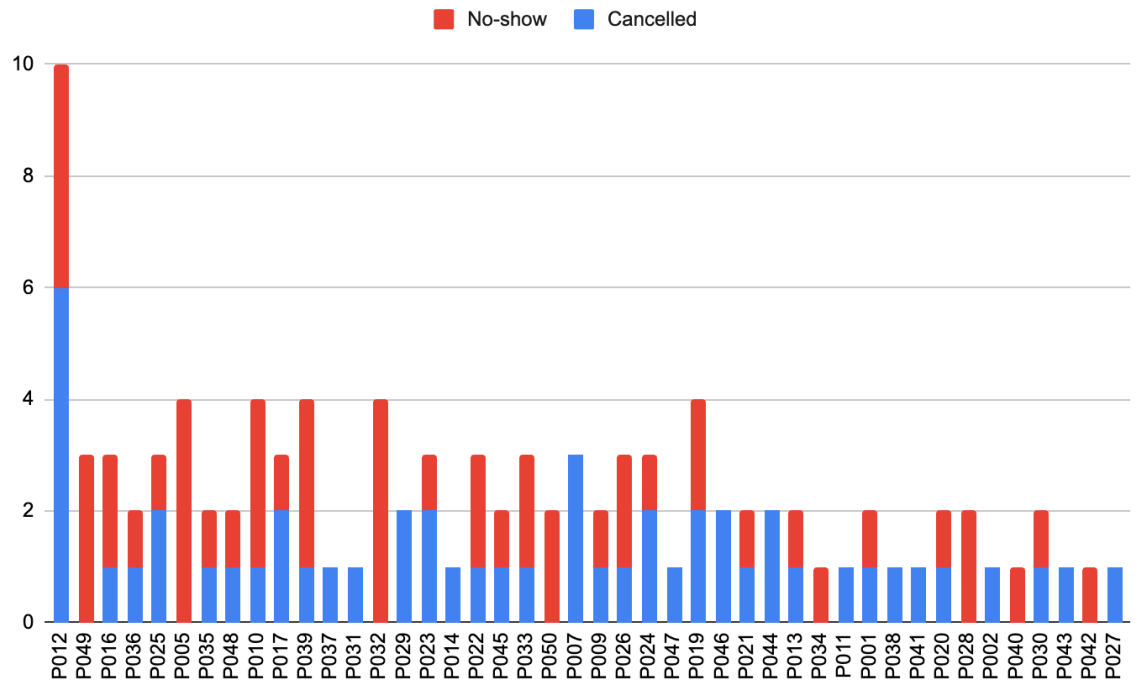

Figure 8

Fraud Detection Logic

To detect fraud, first we needed to check for any anomalies in the dataset. All data links had to be validated in order to make sure there were no anomalies present.

Row	patient_id	appointment_id	treatment_date	bill_date	appointment_date	amount	payment_method	payment_status
1	P001	A048	2023-01-16	2023-01-16	2023-01-16	3249.41	Credit Card	Failed
2	P001	A071	2023-01-26	2023-01-26	2023-01-26	2960.14	Cash	Paid
3	P001	A007	2023-04-09	2023-04-09	2023-04-09	534.03	Cash	Failed
4	P001	A197	2023-04-01	2023-04-01	2023-04-01	975.49	Cash	Pending
5	P002	A082	2023-01-20	2023-01-20	2023-01-20	3615.96	Insurance	Pending
6	P002	A188	2023-04-12	2023-04-12	2023-04-12	616.15	Cash	Paid
7	P002	A055	2023-10-06	2023-10-06	2023-10-06	1736.63	Cash	Failed
8	P003	A013	2023-08-16	2023-08-16	2023-08-16	4704.96	Cash	Paid

Results per page: 50 1 – 50 of 200

Figure 9

We checked data across patients, billing, and treatment by linking treatment, bill, and appointment dates. We ended up with 200 results, which is how many there should be.

Next, we checked to see if there were any duplicates in records. For this we checked email addresses that were used for different patients. In this instance, we filtered the results to show records that have the same email address used more than once.

Row	patient_name	patient_id	patient_email	number_of_records	total_amount_accrued	payment_status
1	Alex Moore	P032	alex.moore@mail.com	2	4958.17	Failed
2	Alex Moore	P028	alex.moore@mail.com	2	894.39	Pending
3	Alex Moore	P028	alex.moore@mail.com	2	2926.23	Failed
4	David Smith	P029	david.smith@mail.com	2	5824.8	Pending
5	David Smith	P038	david.smith@mail.com	2	2691.78	Failed
6	David Smith	P029	david.smith@mail.com	2	1551.7	Failed
7	David Wilson	P035	david.wilson@mail.com	2	7983.0	Pending
8	David Wilson	P005	david.wilson@mail.com	2	3398.12	Pending

Results per page: 50 1 – 35 of 35

Figure 10

Please note: patient information shown, which is to be kept confidential.

Further, from earlier in the report we had noticed there were payments made to patients who had not shown or cancelled their appointments.

Row	patient_id	bills_paid_count
1	P012	6
2	P032	3
3	P019	2
4	P024	2
5	P039	2
6	P049	2
7	P007	2
8	P010	2

Results per page: 50 1 – 8 of 8

Figure 11

From the results we generated, we can see the patients who had cancellation or no-show for their appointments yet they were paid. What does this tell us? Could this be because the billing department is not cross checking the appointment status and approving payments? There could be a problem with the dataset, where this has to be looked into thoroughly.

Actionable Recommendations

In order to ensure we do not get any deviation in our records, the information entered into the dataset must be correct and duplicates are accounted for. In our analysis, we had instances where one email address appeared across multiple patients.

Data validation should be taken into account to ensure data is accurate, reliable, and consistent before being processed. Applying unique constraints on columns that should not have repeated values.

SQL Scripts

```sql
/*
Show total revenue and group them by treatment types
Show each treatment type's total revenue in percentage
*/

WITH
     total_revenue AS (
           SELECT SUM(amount) total_amount
           FROM data-analytics-bootcamp-2026.hospital_management.billing
     ),
     total_revenue_grouped AS (
           SELECT t.treatment_type treatment_type,
                 SUM(b.amount) total_per_type
           FROM data-analytics-bootcamp-2026.hospital_management.billing b
           JOIN data-analytics-bootcamp-2026.hospital_management.treatments t
                 ON b.treatment_id = t.treatment_id
           GROUP BY treatment_type
     )

SELECT treatment_type,
     ROUND(total_amount, 2) total_revenue,
     ROUND(total_per_type, 2) total_revenue_grouped,
     ROUND(((total_per_type * 100)/total_amount), 2) revenue_percent_per_type
FROM total_revenue, total_revenue_grouped
ORDER BY revenue_percent_per_type DESC;
```

SQL Query for Figure 1

```sql
/*
Show total revenue loss, where payment status is failed, and group them by
treatment types
Show each treatment type's total revenue loss in percentage
*/

WITH
     total_revenue_loss AS (
           SELECT SUM(amount) total_loss_amount
```

```sql
            FROM data-analytics-bootcamp-2026.hospital_management.billing
            WHERE UPPER(payment_status) IN ('FAILED')
      ),
      total_revenue_grouped AS (
            SELECT t.treatment_type treatment_type,
                  SUM(b.amount) total_per_type
            FROM data-analytics-bootcamp-2026.hospital_management.billing b
            JOIN data-analytics-bootcamp-2026.hospital_management.treatments t
                  ON b.treatment_id = t.treatment_id
            WHERE UPPER(payment_status) IN ('FAILED')
            GROUP BY treatment_type
      )

SELECT treatment_type,
      ROUND(total_loss_amount, 2) total_loss_revenue,
      ROUND(total_per_type, 2) total_revenue_grouped,
      ROUND(((total_per_type * 100)/total_loss_amount), 2)
revenue_percent_per_type
FROM total_revenue_loss, total_revenue_grouped
ORDER BY revenue_percent_per_type DESC;
```

SQL Query for Figure 2

```sql
/*
Find total loss revenue in percentage
*/

WITH
      total_revenue_loss AS (
            SELECT ROUND(SUM(amount), 2) total_loss_amount
            FROM data-analytics-bootcamp-2026.hospital_management.billing
            WHERE UPPER(payment_status) IN ('FAILED')
      ),
      total_revenue AS (
            SELECT ROUND(SUM(amount), 2) total_amount
            FROM data-analytics-bootcamp-2026.hospital_management.billing
      )

SELECT ROUND(SAFE_DIVIDE(trl.total_loss_amount * 100, tr.total_amount), 2)
```

```
lost_revenue_percent
FROM total_revenue_loss trl, total_revenue tr;
```

SQL Query for Figure 3

```
/*
Show the total count of Cancelled of No-show appointment under one category and
show the count of Completed and Scheduled separately
*/

WITH
   status_group AS (
         SELECT status appt_status,
               CASE
                   WHEN UPPER(status) IN ('CANCELLED','NO-SHOW') THEN 'Cancelled
or No-Show'
                   WHEN UPPER(status) IN ('COMPLETED') THEN 'Completed'
                   ELSE 'Scheduled'
               END status_categorised
         FROM data-analytics-bootcamp-2026.hospital_management.appointments
   )

SELECT status_categorised,
     COUNT(status_categorised) status_count
FROM status_group
GROUP BY status_categorised
```

SQL Query for Figure 4

```
/*
Show the total count of Cancelled of No-show appointment under one category and
show the count of Completed and Scheduled separately
*/

WITH
   status_group AS (
         SELECT status appt_status,
               CASE
                   WHEN UPPER(status) IN ('CANCELLED','NO-SHOW') THEN 'Cancelled
```

```
or No-Show'
                    WHEN UPPER(status) IN ('COMPLETED') THEN 'Completed'
                    ELSE 'Scheduled'
                END status_categorised
          FROM data-analytics-bootcamp-2026.hospital_management.appointments
    )

SELECT status_categorised,
     COUNT(status_categorised) status_count
FROM status_group
GROUP BY status_categorised
```

SQL Query for Figure 5

```
/*
Sorting patients by gender
*/

SELECT p.gender,
     COUNT(p.gender) total,
     ROUND(COUNT(*) * 100.0 / (SELECT COUNT(*) FROM
data-analytics-bootcamp-2026.hospital_management.patients), 2) percentage
FROM data-analytics-bootcamp-2026.hospital_management.patients p
GROUP BY p.gender;
```

SQL Query for Figure 6

```
/*
Sorting patients into age groups (18-24, 25-34, 35-44, 45-54. 55-64. 65+).
Assuming all patients are 18+
*/

WITH age_group_bracket AS (
     SELECT DISTINCT(p.patient_id),
           CASE
                 WHEN DIV(DATE_DIFF(MAX(a.appointment_date), p.date_of_birth,
day), 365) BETWEEN 18 AND 24 THEN '18 - 24'
                 WHEN DIV(DATE_DIFF(MAX(a.appointment_date), p.date_of_birth,
day), 365) BETWEEN 25 AND 34 THEN '25 - 34'
```

```
                WHEN DIV(DATE_DIFF(MAX(a.appointment_date), p.date_of_birth,
day), 365) BETWEEN 35 AND 44 THEN '35 - 44'
                WHEN DIV(DATE_DIFF(MAX(a.appointment_date), p.date_of_birth,
day), 365) BETWEEN 45 AND 54 THEN '45 - 54'
                WHEN DIV(DATE_DIFF(MAX(a.appointment_date), p.date_of_birth,
day), 365) BETWEEN 55 AND 64 THEN '55 - 64'
                ELSE '65+'
           END age_group
     FROM data-analytics-bootcamp-2026.hospital_management.patients p
     LEFT JOIN data-analytics-bootcamp-2026.hospital_management.appointments a
           ON p.patient_id = a.patient_id
     GROUP BY p.patient_id, p.date_of_birth
     ORDER BY age_group
)

SELECT agb.age_group,
     COUNT(agb.age_group) total
FROM age_group_bracket agb
GROUP BY agb.age_group
ORDER BY agb.age_group;
```

SQL Query for Figure 7

```
/*
Show all patients who have at least one cancelled or no-show appointment status.
Also show the total amount they have accrued.
*/

WITH
total_per_patient AS (
     SELECT patient_id patient_id,
     ROUND(SUM(amount), 2) total_amount
     FROM data-analytics-bootcamp-2026.hospital_management.billing
     GROUP BY patient_id
),
appt_status AS (
     SELECT patient_id patient_id,
           COUNT(*) total_appointments,
           COUNTIF(UPPER(status) = 'COMPLETED') completed,
```

```sql
            COUNTIF(UPPER(status) = 'SCHEDULED') scheduled,
            COUNTIF(UPPER(status) = 'CANCELLED') cancelled,
            COUNTIF(UPPER(status) = 'NO-SHOW') no_show,
      FROM data-analytics-bootcamp-2026.hospital_management.appointments
      GROUP BY patient_id
      ORDER BY cancelled DESC, no_show DESC
)

SELECT tpp.patient_id,
      tpp.total_amount,
      astat.total_appointments,
      astat.completed,
      astat.scheduled,
      astat.cancelled,
      astat.no_show
FROM total_per_patient tpp
JOIN appt_status astat
      ON tpp.patient_id = astat.patient_id
WHERE astat.cancelled > 0 OR astat.no_show > 0
ORDER BY total_amount DESC;
```

SQL Query for Figure 8

```sql
/*
Checking anomalies across treatments, billing, and appointments tables to pick on
anything unusual.
The results show 200 records.
*/


SELECT b.patient_id,
      a.appointment_id,
      t.treatment_date,
      b.bill_date,
      a.appointment_date,
      b.amount,
      b.payment_method,
      b.payment_status
```

```sql
FROM data-analytics-bootcamp-2026.hospital_management.treatments t
JOIN data-analytics-bootcamp-2026.hospital_management.billing b
     ON t.treatment_id = b.treatment_id
JOIN data-analytics-bootcamp-2026.hospital_management.appointments a
     ON a.appointment_id = t.appointment_id
WHERE DATE(b.bill_date) = DATE(t.treatment_date) AND DATE(t.treatment_date) =
DATE(a.appointment_date)
ORDER BY a.patient_id;
```

SQL Query for Figure 9

```sql
/*
Show records where patient details appear more than once, where email address is
used across multiple patients.
Only show records that have an email address appearing more than once.
*/


WITH duplicate_records AS (
 SELECT CONCAT(p.first_name, ' ', p.last_name) patient_name,
       p.email patient_email,
       COUNT(p.patient_id) number_of_records
 FROM data-analytics-bootcamp-2026.hospital_management.patients p
 GROUP BY patient_name, patient_email
 HAVING COUNT(p.patient_id) > 1
)

SELECT duplicate.patient_name,
     original_patient.patient_id,
     duplicate.patient_email,
     duplicate.number_of_records,
     ROUND(SUM(b.amount), 2) total_amount_accrued,
     b.payment_status payment_status
FROM duplicate_records duplicate
JOIN data-analytics-bootcamp-2026.hospital_management.patients original_patient
     ON duplicate.patient_email = original_patient.email
JOIN data-analytics-bootcamp-2026.hospital_management.billing b
     ON b.patient_id = original_patient.patient_id
WHERE UPPER(payment_status) IN ('FAILED', 'PENDING')
```

```sql
GROUP BY duplicate.patient_name, original_patient.patient_id,
duplicate.patient_email, duplicate.number_of_records, payment_status
ORDER BY duplicate.patient_name;
```

SQL Query for Figure 10

```sql
/*
List patients and the total count of bills they have been paid.
This is to flag patients who have incurred a lot of bills
*/

SELECT b.patient_id, COUNT(b.bill_id) bills_paid_count
FROM data-analytics-bootcamp-2026.hospital_management.billing b
JOIN data-analytics-bootcamp-2026.hospital_management.treatments t
     ON b.treatment_id = t.treatment_id
WHERE appointment_id IN (SELECT appointment_id
                         FROM
data-analytics-bootcamp-2026.hospital_management.appointments
                         WHERE UPPER(status) IN ('CANCELLED','NO-SHOW'))
GROUP BY b.patient_id, b.payment_status
HAVING UPPER(b.payment_status) IN ('PAID') AND COUNT(b.bill_id) > 1
ORDER BY bills_paid_count DESC
```

SQL Query for Figure 11